INSTITUT UNIVERSITAIRE DES SCIENCES

(**IUS**)

Faculté des Sciences et Technologies

(FST)

TD Nº 3 – Modélisation

Nom & Prénom :

BICHOTTE Kendy

Niveau :

L3

Date : Avril 2026

Introduction

Ce TP mobilise trois outils essentiels de la modélisation mathématique : les matrices, les graphes et les chaînes de Markov. À travers des applications variées : réseaux, transport, processus probabilistes. Il vise à montrer comment ces structures permettent de représenter, analyser et optimiser des systèmes complexes.

TD 1 : Réseau social intelligent :

Construire un graphe représentant des utilisateurs

- sommets = utilisateurs
- arêtes = relations

1. créer un réseau de 20 utilisateurs

TD1-TP3.py

TD3 > TD1-TP3.py > ...

```python
import networkx as nx
import matplotlib.pyplot as plt
from networkx.algorithms.community import greedy_modularity_communities


G = nx.Graph()

# Création des utilisateurs U1 à U20
users = [f"U{i}" for i in range(1, 21)]
G.add_nodes_from(users)

#  Communauté A : U1 à U7
comm_A = [
    ("U1","U2"),("U1","U3"),("U1","U4"),
    ("U2","U3"),("U2","U5"),
    ("U3","U6"),("U4","U5"),
    ("U5","U6"),("U6","U7"),("U4","U7")
]

#  Communauté B : U8 à U14
comm_B = [
    ("U8","U9"),("U8","U10"),
    ("U9","U10"),("U9","U11"),
    ("U10","U12"),("U11","U12"),
    ("U11","U13"),("U12","U14"),
    ("U13","U14")
]

#  Communauté C : U15 à U20
comm_C = [
    ("U15","U16"),("U15","U17"),
    ("U16","U17"),("U16","U18"),
```

Ln 37, Col 3 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Python 3.14 (64-bit) Go Live Prettier

```python
comm_C = [
    ("U15","U16"),("U15","U17"),
    ("U16","U17"),("U16","U18"),
    ("U17","U19"),("U18","U19"),
    ("U18","U20"),("U19","U20")
]

# Ponts entre communautés
bridges = [
    ("U3","U9"),
    ("U6","U12"),
    ("U11","U16")
]

# Ajout de toutes les relations
G.add_edges_from(comm_A + comm_B + comm_C + bridges)
```

2. Trouver les personnes les plus influentes

```python
# 2. Personnes les plus influentes (centralité de degré)
degrees = dict(G.degree())
max_degree = max(degrees.values())

influents = [u for u, d in degrees.items() if d == max_degree]

print("=== Utilisateurs les plus influents ===")
print(f"Degré maximal : {max_degree}")
print("Influents :", influents)
```

3. Calculer le degré de chaque nœud

```python
# 3. Calcul du degré de chaque utilisateur
print("\n=== Degré de chaque utilisateur ===")
for user, deg in degrees.items():
    print(f"{user} → degré = {deg}")
```

4. détecter les communautés

```python
# 4. Détection des communautés (méthode greedy)
communities = greedy_modularity_communities(G)
print("\n=== Communautés détectées ===")
for i, comm in enumerate(communities, start=1):
    print(f"Communauté {i} :", sorted(list(comm)))
```

Code de visualisation du graphe

```python
# 5. Visualisation du graphe
plt.figure(figsize=(10, 7))
pos = nx.spring_layout(G, seed=42)

nx.draw(
    G, pos,
    with_labels=True,
    node_color="blue",
    node_size=600,
    font_size=9,
    edge_color="black"
)

plt.title("TD1 – Réseau social intelligent")
plt.show()
```

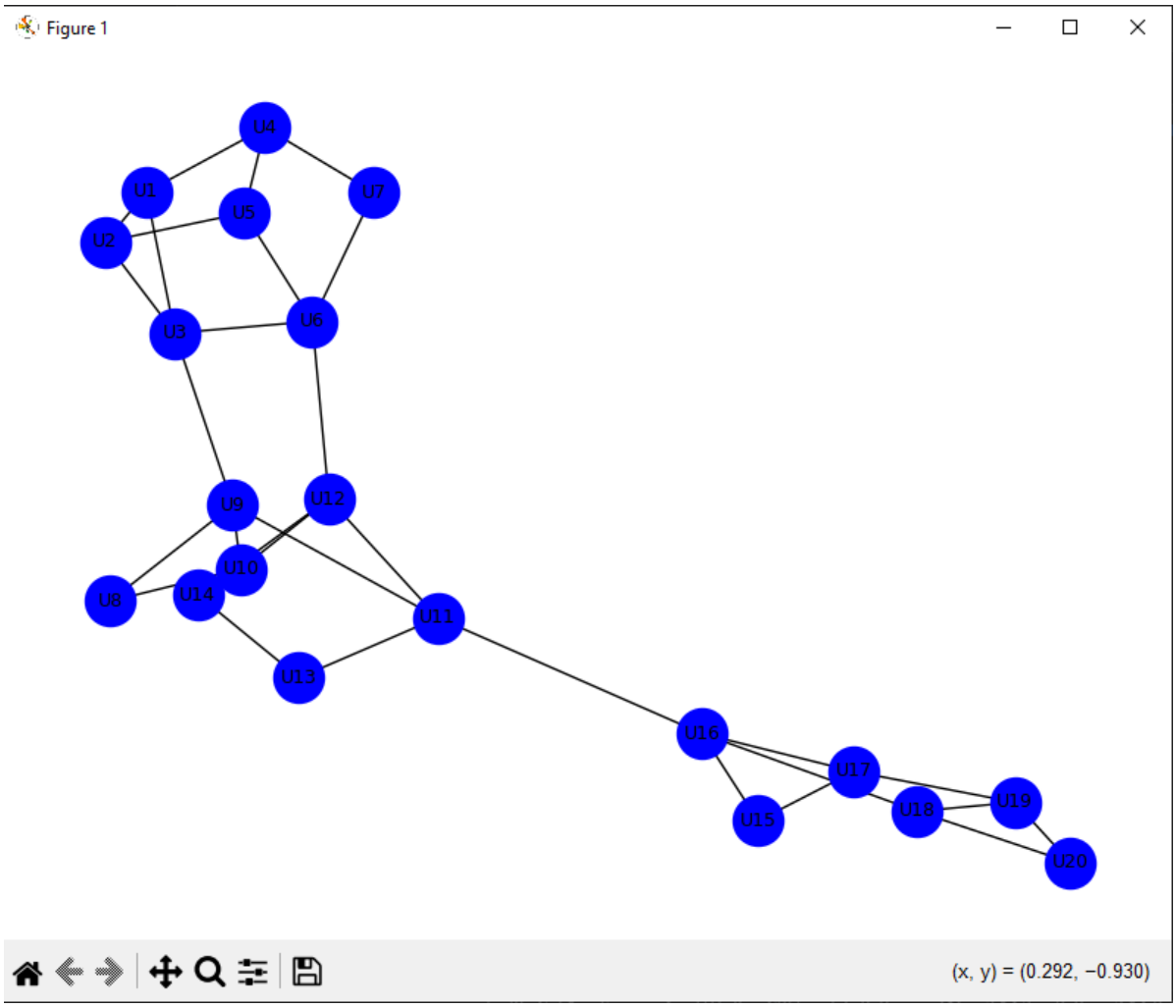
Résultat du code exécuté

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                  Python

PS C:\Users\afjte\Downloads\Modelisation> & C:/Users/afjte/AppData/Local/Programs/Python/Python314/python.exe c:/Users/afjte/Downloads/Model
isation/TD3/TD1-TP3.py
=== Utilisateurs les plus influents ===
Degré maximal : 4
Influents : ['U3', 'U6', 'U9', 'U11', 'U12', 'U16']

=== Degré de chaque utilisateur ===
U1 → degré = 3
U2 → degré = 3
U3 → degré = 4
U4 → degré = 3
U5 → degré = 3
U6 → degré = 4
U7 → degré = 2
U8 → degré = 2
U9 → degré = 4
U10 → degré = 3
U11 → degré = 4
U12 → degré = 4
U13 → degré = 2
U14 → degré = 2
U15 → degré = 2
U16 → degré = 4
U17 → degré = 3
U18 → degré = 3
U19 → degré = 3
U20 → degré = 2

=== Communautés détectées ===
Communauté 1 : ['U1', 'U2', 'U3', 'U4', 'U5', 'U6', 'U7']
Communauté 2 : ['U10', 'U11', 'U12', 'U13', 'U14', 'U8', 'U9']
Communauté 3 : ['U15', 'U16', 'U17', 'U18', 'U19', 'U20']
```



TD 2: Chaîne de Markov (météo + économie)

Modéliser deux systèmes :

1. météo

- Soleil
- Pluie
- Nuage

2. économie

- Hausse
- Stable
- Baisse

Objectifs

1. Construire la matrice de transition
2. Simuler 50 à 100 jours
3. Calculer la distribution stationnaire
4. Comparer simulation vs théorie

Modéliser deux systèmes

1-Construire la matrice de transition

TD2-TP3.py

TD3 > TD2-TP3.py > ...

```python
import numpy as np

# MODELISATION DES DEUX SYSTEMES
# --- Système météo ---
etats_meteo = ["Soleil", "Pluie", "Nuage"]

# Matrice de transition météo
P_meteo = np.array([
    [0.7, 0.2, 0.1],   # Soleil : Soleil, Pluie, Nuage
    [0.4, 0.4, 0.2],   # Pluie  : Soleil, Pluie, Nuage
    [0.3, 0.3, 0.4]    # Nuage  : Soleil, Pluie, Nuage
])

# --- Système économie ---
etats_eco = ["Hausse", "Stable", "Baisse"]

# Matrice de transition économie
P_eco = np.array([
    [0.6, 0.3, 0.1],   # Hausse : Hausse, Stable, Baisse
    [0.2, 0.6, 0.2],   # Stable : Hausse, Stable, Baisse
    [0.1, 0.3, 0.6]    # Baisse : Hausse, Stable, Baisse
])
```

```python

# FONCTIONS GENERIQUES
def simuler(P, X0, n):
    X = X0.copy()
    for _ in range(n):
        X = X @ P
    return X

def stationnaire(P):
    w, v = np.linalg.eig(P.T)
    i = np.argmin(np.abs(w - 1))
    T = v[:, i].real
    return T / T.sum()

def afficher(systeme, etats, Xn, T):
    print(f"\n=== {systeme} ===")
    print("États :", etats)
    print("Après 50 jours :", Xn.round(4))
    print("Stationnaire   :", T.round(4))
    print("Différence     :", (Xn - T).round(6))
```

Simulation et distribution stationnaire

```python
# SIMULATION + DISTRIBUTION STATIONNAIRE
# Météo
X0_meteo = np.array([1, 0, 0])  # Soleil au jour 0
Xn_meteo = simuler(P_meteo, X0_meteo, 50)
T_meteo = stationnaire(P_meteo)

# Économie
X0_eco = np.array([0.5, 0.3, 0.2])  # Distribution initiale
Xn_eco = simuler(P_eco, X0_eco, 50)
T_eco = stationnaire(P_eco)
```

Comparaison simulation vs théorie

```python
#COMPARAISON SIMULATION VS THEORIE
afficher("Météo", etats_meteo, Xn_meteo, T_meteo)
afficher("Économie", etats_eco, Xn_eco, T_eco)
print("\nConclusion : les distributions simulées convergent vers les distributions stationnaires.")
```

Résultat

```
PS C:\Users\afjte\Downloads\Modelisation> & C:/Users/afjte/AppData/Local/Programs/Python/Python314/python.exe c:/Users/afjte/Downloads/Model
isation/TD3/TD2-TP3.py

=== Météo ===
États : ['Soleil', 'Pluie', 'Nuage']
Après 50 jours : [0.5455 0.2727 0.1818]
Stationnaire   : [0.5455 0.2727 0.1818]
Différence     : [ 0. -0. -0.]

=== Économie ===
États : ['Hausse', 'Stable', 'Baisse']
Après 50 jours : [0.2857 0.4286 0.2857]
Stationnaire   : [0.2857 0.4286 0.2857]
Différence     : [-0. -0. -0.]

Conclusion : les distributions simulées convergent vers les distributions stationnaires.
PS C:\Users\afjte\Downloads\Modelisation>
```

TD3 : Réseau de transport (type métro)

Créer un réseau de métro simplifié.

Objectifs

1. Créer 15 à 25 stations
2. Relier les stations
3. Associer des temps de trajet
4. Calculer itinéraires optimaux
5. Afficher les lignes

Création, liaison des stations et association des temps de trajet

```python
import networkx as nx
import matplotlib.pyplot as plt

# Initialisation du graphe
metro = nx.Graph()

# Création des stations, liens et temps de trajet

# Ligne Nord
ligne_nord = [
    ("Ouanaminthe", "Fort-Liberté", 15),
    ("Fort-Liberté", "Limonade", 12),
    ("Limonade", "Cap-Haïtien", 10),
    ("Cap-Haïtien", "Gonaïves", 45),
    ("Gonaïves", "Saint-Marc", 30),
    ("Saint-Marc", "Port-au-Prince", 40)
]

# Ligne Sud
ligne_sud = [
    ("Port-au-Prince", "Carrefour", 10),
    ("Carrefour", "Léogâne", 15),
    ("Léogâne", "Grand-Goâve", 10),
    ("Grand-Goâve", "Petit-Goâve", 8),
    ("Petit-Goâve", "Miragoâne", 12),
    ("Miragoâne", "Les Cayes", 40),
    ("Les Cayes", "Port-Salut", 15),
    ("Les Cayes", "Jérémie", 60)
]
# Ligne Est/Centre
ligne_est = [
    ("Belladère", "Hinche", 25),
    ("Hinche", "Mirebalais", 20),
    ("Mirebalais", "Port-au-Prince", 35),
    ("Port-au-Prince", "Pétion-Ville", 15),
    ("Pétion-Ville", "Kenscoff", 12),
    ("Pétion-Ville", "Jacmel", 50)
]
```

```python
# Ajout des lignes au graphe
for ligne in [ligne_nord, ligne_sud, ligne_est]:
    metro.add_weighted_edges_from(ligne)
```

Calcul de l'itinéraire optimal

```python
# Calcul de l'itinéraire optimal

def calculer_trajet(depart, arrivee):
    try:
        # Utilisation de l'algorithme de Dijkstra (poids = temps)
        temps_total = nx.shortest_path_length(metro, source=depart, target=arrivee, weight='weight')
        chemin = nx.shortest_path(metro, source=depart, target=arrivee, weight='weight')

        print(f" Trajet de '{depart}' à '{arrivee}' :")
        print(f" Temps estimé : {temps_total} minutes")
        print(f" Itinéraire : {' → '.join(chemin)}")
    except nx.NetworkXNoPath:
        print("Aucun chemin trouvé.")
```

Affichage des lignes

```python
# Affichage des lignes
print(f"Nombre de stations créées : {metro.number_of_nodes()}")
calculer_trajet("Port-au-Prince", "Jacmel")
```

Visualisation du graphe

```python
# Visualisation graphique
plt.figure(figsize=(12, 9))
pos = nx.spring_layout(metro)
nx.draw(metro, pos, with_labels=True, node_color='skyblue', node_size=2000, font_size=8)
labels = nx.get_edge_attributes(metro, 'weight')
nx.draw_networkx_edge_labels(metro, pos, edge_labels=labels)
plt.title("Plan du Réseau de Métro Simplifié")
plt.show()
```

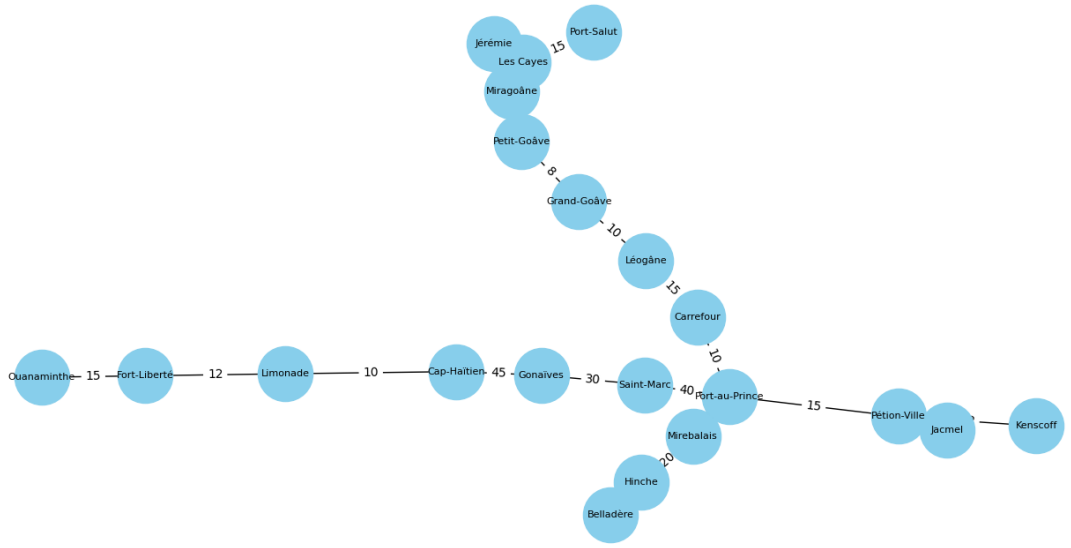
Résultat du code exécuté

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Python

Nombre de stations créées : 21
 Trajet de 'Port-au-Prince' à 'Jacmel' :
 Temps estimé : 65 minutes
 Itinéraire : Port-au-Prince →Pétion-Ville →Jacmel
```

Figure 1- 21 stations ont été créés. Comme on peut le remarquer : le trajet Port-au-Prince....Pétion-ville est de 15 minutes ; celui de Pétion-Ville....Jacmel est de 50 minutes. Le temps estimé est donc de 65 minutes pour l'itinéraire.

```python
# Ligne Est/Centre
ligne_est = [
    ("Belladère", "Hinche", 25),
    ("Hinche", "Mirebalais", 20),
    ("Mirebalais", "Port-au-Prince", 35),
    ("Port-au-Prince", "Pétion-Ville", 15),
    ("Pétion-Ville", "Kenscoff", 12),
    ("Pétion-Ville", "Jacmel", 50)
]
```



Conclusion

L'ensemble du TP met en évidence la complémentarité entre matrices, graphes et chaînes de Markov dans la modélisation de systèmes réels. Les activités réalisées démontrent la capacité de ces outils à structurer l'information, à analyser des relations complexes et à simuler des évolutions dynamiques. Les trois TD permettent d'approfondir ces notions à travers des applications concrètes, allant de l'analyse de réseaux sociaux à la modélisation économique et au calcul d'itinéraires optimaux. Ce travail confirme ainsi l'importance de ces approches mathématiques et algorithmiques dans la compréhension et l'optimisation de systèmes modernes.